AWS Scaling Design Analysis LAB

Deepika Agnihotri
University of Maryland Global Campus
Cloud Infrastructure Planning and Design & CLCS 645
Professor Loye Ray

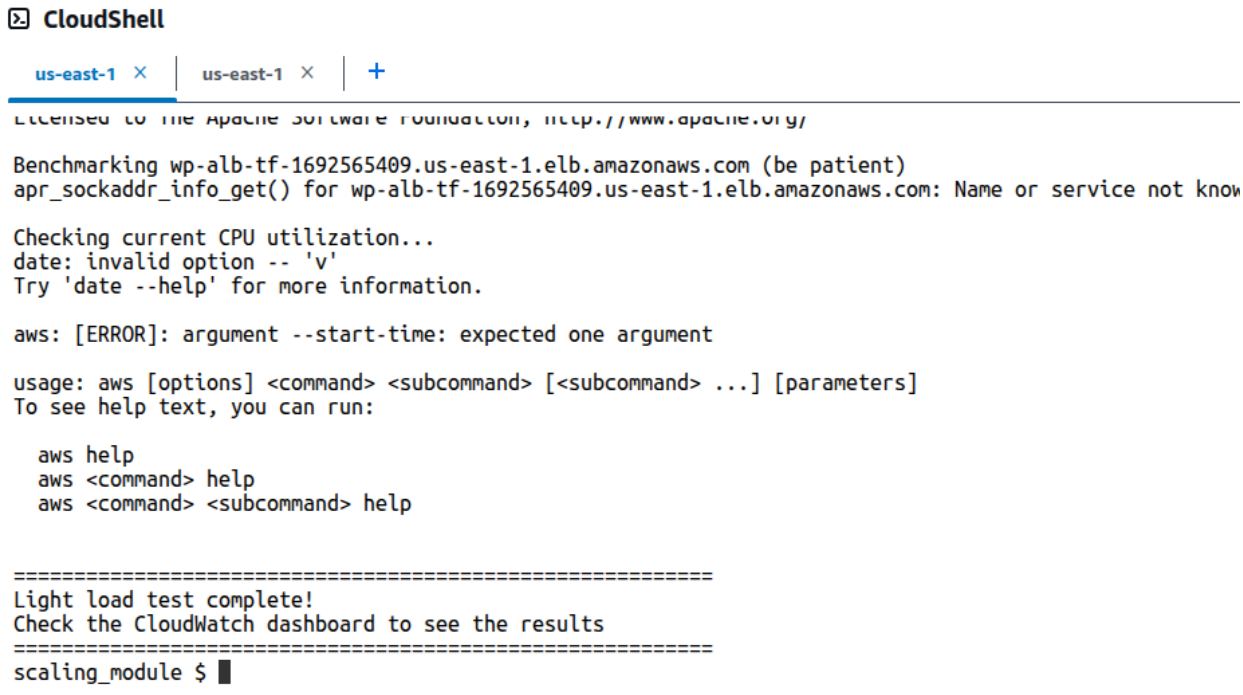
This report summarizes the results of performance testing conducted on the system across multiple load scenarios, including light load, medium load, heavy load, burst load, gradual ramp tests, and quick load tests. The objective is to evaluate system stability, responsiveness, throughput, and resource consumption under various stress levels.

Testing Methodology and Scenarios

Performance testing followed an incremental-load methodology to observe how the system behaves as demand increases. Each scenario targeted a specific pattern of user activity.

Light Load Test

The purpose of this test is to validate system performance under minimal expected user activity. The scenario involves 10–50 virtual users with standard request rates during a short-duration test of 10–20 minutes. The focus areas include basic responsiveness, error-free operation, and monitoring CPU and memory usage at idle-to-low utilization.



Monitoring light load

CloudShell

us-east-1 × | us-east-1 × | +

```
============================================================
WordPress ECS Service Instance Monitor
Refreshing every 30 seconds. Press Ctrl+C to stop.
============================================================\033[0m
Check #13 - 2025-12-01 08:16:27

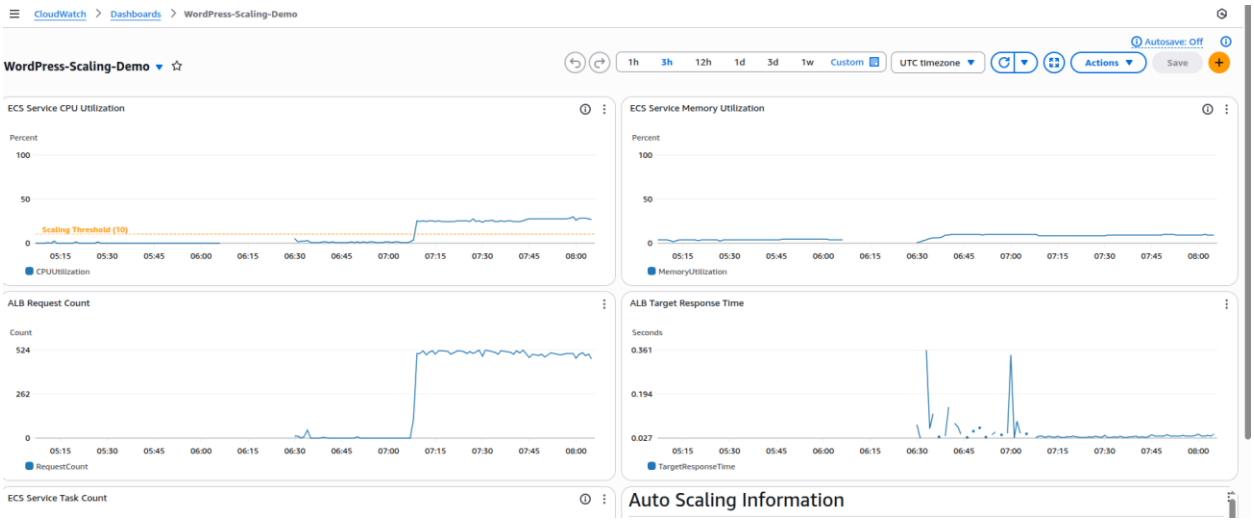
Instance Count:
  Running: 2
  Desired: 2
  Status: Stable

CPU Utilization:
  Current: 0.00%
  Threshold: 10% (Auto-scaling trigger point)
./monitor_instances.sh: line 100: bc: command not found
  Analysis: Below threshold - No scaling needed

Command used:
aws ecs describe-services --cluster ecs-wordpress --services wp-ecs-svc-tf --region us-east-1

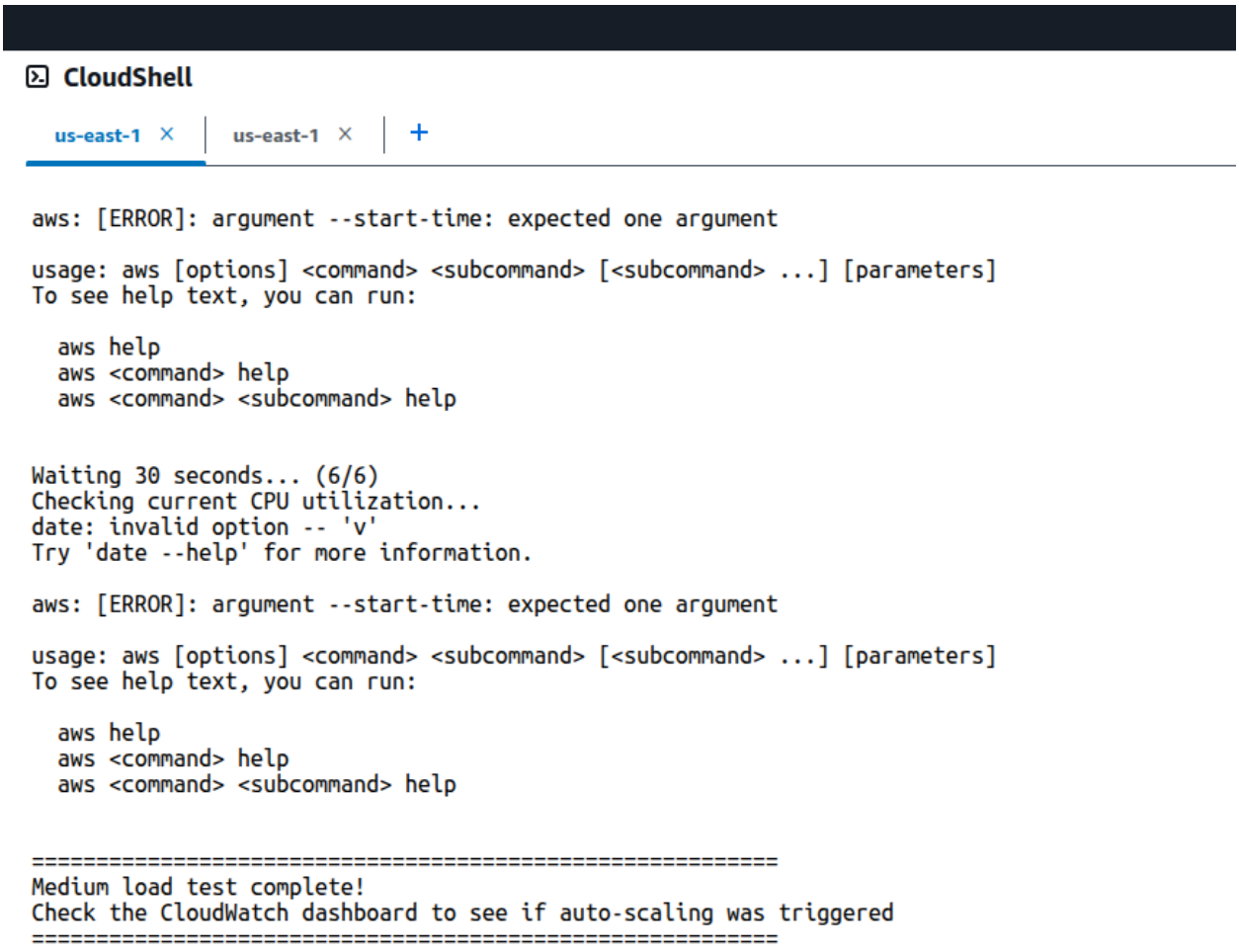
Waiting 30 seconds for next check...
```

Cloud watch image of light load:



Medium Load Test

The purpose of the Medium Load Test is to measure performance near expected average daily traffic. The scenario includes 100–200 virtual users with a realistic request for distribution over a 30–60-minute duration. The focus areas are sustained throughput, API latency consistency, and overall service-level stability.


CloudShell
us-east-1
us-east-1
aws: [ERROR]: argument --start-time: expected one argument
usage: aws [options] <command> <subcommand> [<subcommand> ...] [parameters]
To see help text, you can run:
aws help
aws <command> help
aws <command> <subcommand> help
Waiting 30 seconds... (6/6)
Checking current CPU utilization...
date: invalid option -- 'v'
Try 'date --help' for more information.
aws: [ERROR]: argument --start-time: expected one argument
usage: aws [options] <command> <subcommand> [<subcommand> ...] [parameters]
To see help text, you can run:
aws help
aws <command> help
aws <command> <subcommand> help
Medium load test complete!
Check the CloudWatch dashboard to see if auto-scaling was triggered

Monitoring:

CloudShell

us-east-1 × | us-east-1 × | +

```
============================================================\033[0m
============================================================
WordPress ECS Service Instance Monitor
Refreshing every 30 seconds. Press Ctrl+C to stop.
============================================================\033[0m
Check #30 - 2025-12-01 08:25:20

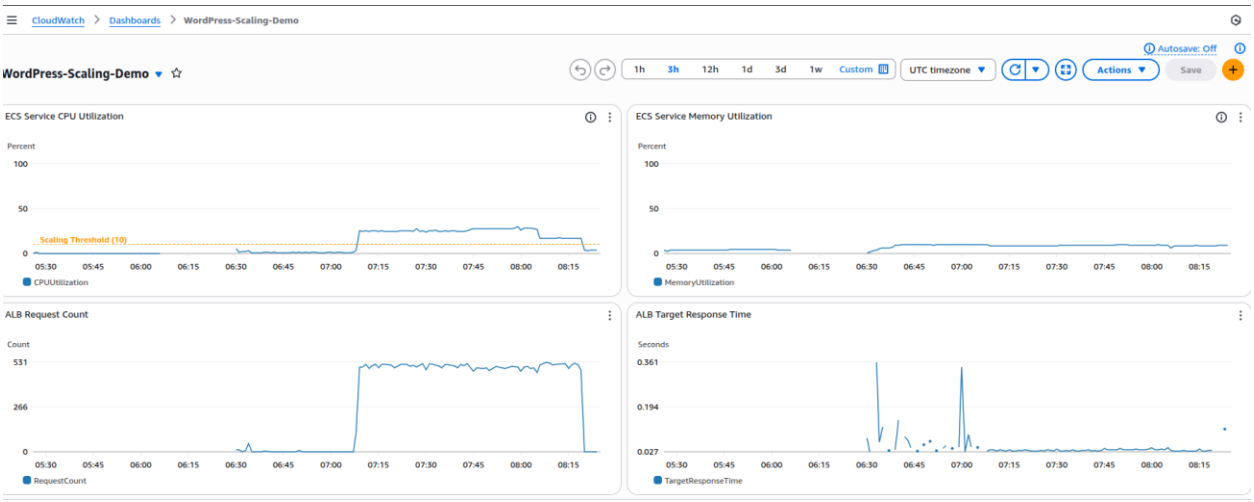
Instance Count:
  Running: 2
  Desired: 2
  Status: Stable

CPU Utilization:
  Current: 0.00%
  Threshold: 10% (Auto-scaling trigger point)
./monitor_instances.sh: line 100: bc: command not found
  Analysis: Below threshold - No scaling needed

Command used:
aws ecs describe-services --cluster ecs-wordpress --services wp-ecs-svc-tf --region us-east-1

Waiting 30 seconds for next check...
```

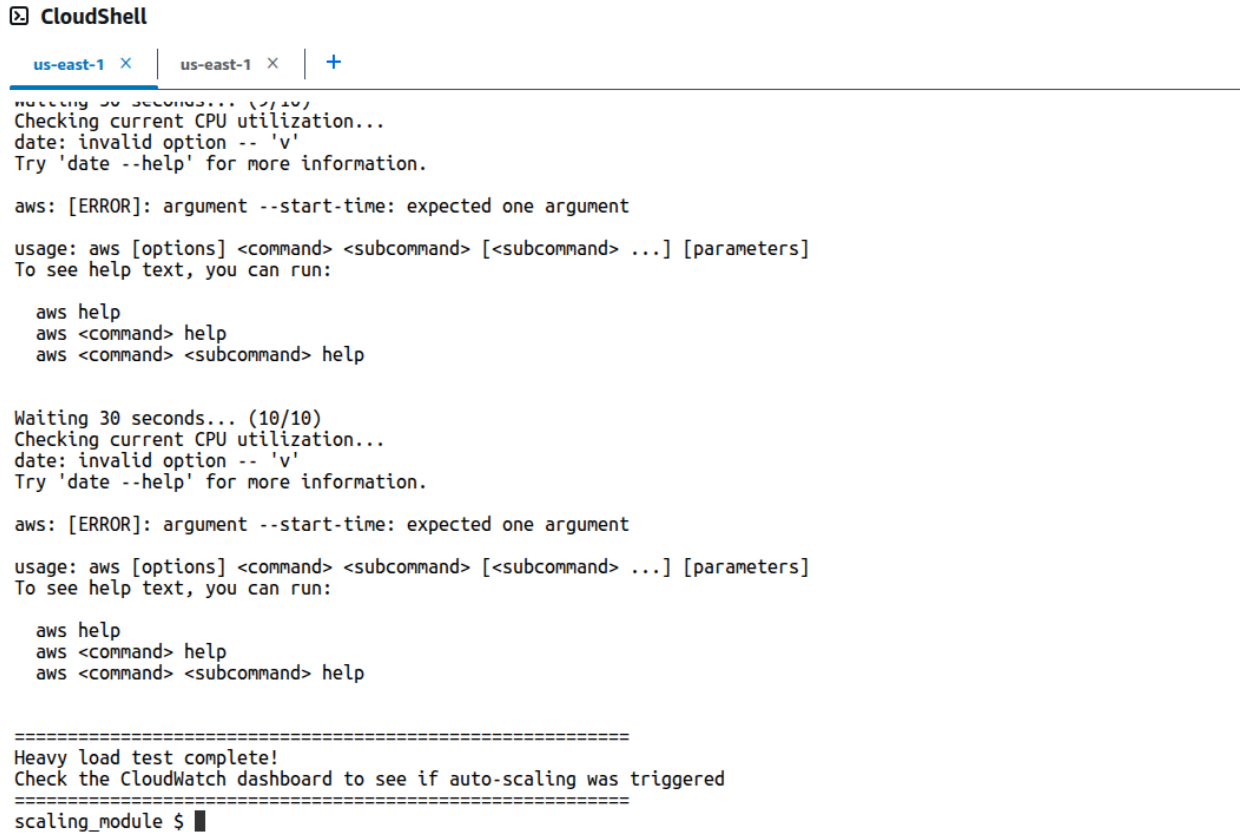
Cloud watch image



Heavy Load Test

The purpose of the Heavy Load Test is to determine system behavior near its upper operational limits. The scenario involves 400–600 virtual users performing high read/write operations over a 60–90-minute duration. The focus areas include observing latency spikes, monitoring error rates

under strain, and evaluating CPU saturation and memory pressure.



Monitoring image of heavy load

CloudShell

us-east-1 | us-east-1 | +

```
============================================================
WordPress ECS Service Instance Monitor
Refreshing every 30 seconds. Press Ctrl+C to stop.
============================================================\033[0m
Check #54 - 2025-12-01 08:37:52

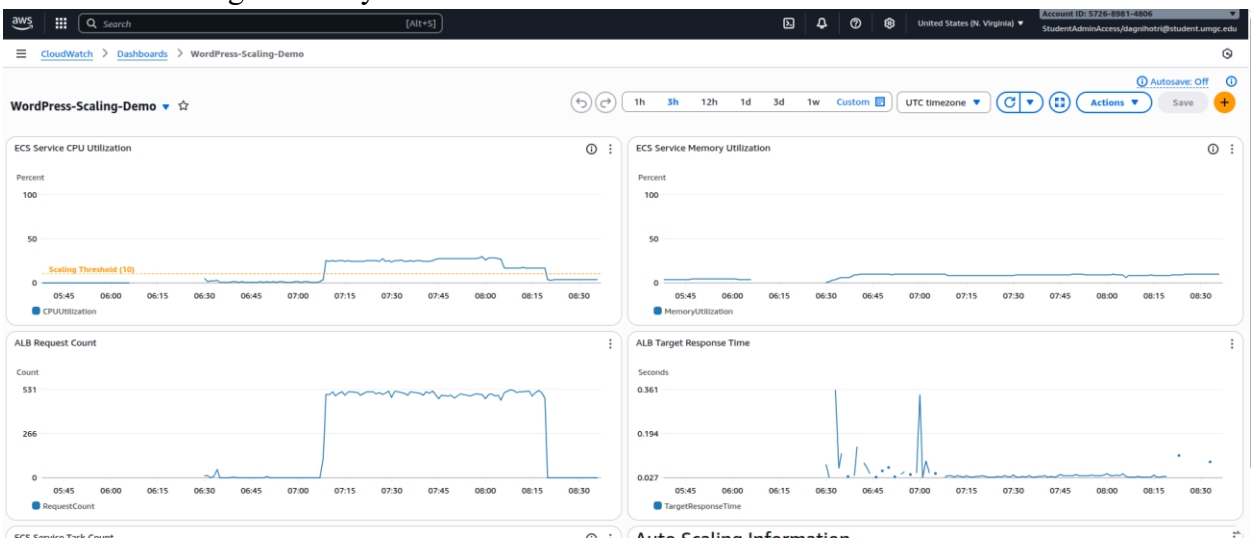
Instance Count:
  Running: 1
  Desired: 1
  Status: Stable

CPU Utilization:
  Current: 0.00%
  Threshold: 10% (Auto-scaling trigger point)
./monitor_instances.sh: line 100: bc: command not found
  Analysis: Below threshold - No scaling needed

Command used:
aws ecs describe-services --cluster ecs-wordpress --services wp-ecs-svc-tf --region us-east-1

Waiting 30 seconds for next check...
```

Cloud Watch image of heavy load



Burst Load Test

The purpose of the Burst Load Test is to measure resilience during abrupt, high-intensity usage spikes. The scenario involves a sudden jump from 50 to 300 users within seconds, executed in repeated burst intervals. The focus areas include auto-scaling responsiveness, queue handling, and overall system recovery time.

CloudShell

Actions

us-east-1 | us-east-1 | +

```
This is ApacheBench, Version 2.3 <$Revision: 1923142 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking wp-alb-tf-1692565409.us-east-1.elb.amazonaws.com (be patient)
This is ApacheBench, Version 2.3 <$Revision: 1923142 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking wp-alb-tf-1692565409.us-east-1.elb.amazonaws.com (be patient)
apr_sockaddr_info_get() for wp-alb-tf-1692565409.us-east-1.elb.amazonaws.com: Name or service not known (670002)
apr_sockaddr_info_get() for wp-alb-tf-1692565409.us-east-1.elb.amazonaws.com: Name or service not known (670002)
apr_sockaddr_info_get() for wp-alb-tf-1692565409.us-east-1.elb.amazonaws.com: Name or service not known (670002)

Checking running instances...
-----------------------
|   DescribeServices   |
+----------+-----------+
| Desired |  Running  |
+----------+-----------+
| 1       | 1         |
+----------+-----------+

============================================================
Burst load test complete!
Check the CloudWatch dashboard to see if auto-scaling was triggered
============================================================
To continuously monitor instances, run:
watch -n 5 'aws ecs describe-services --cluster ecs-wordpress --services wp-ecs-svc-tf --region us-east-1 --query "services[0].{Running:runningCount,Desired:desiredCount}" --output table'
scaling_module $
```

Monitor heavy load

CloudShell

us-east-1 | us-east-1 | +

```
Check #58 - 2025-12-01 08:39:57
============================================================
WordPress ECS Service Instance Monitor
Refreshing every 30 seconds. Press Ctrl+C to stop.
============================================================\033[0m
Check #59 - 2025-12-01 08:40:29

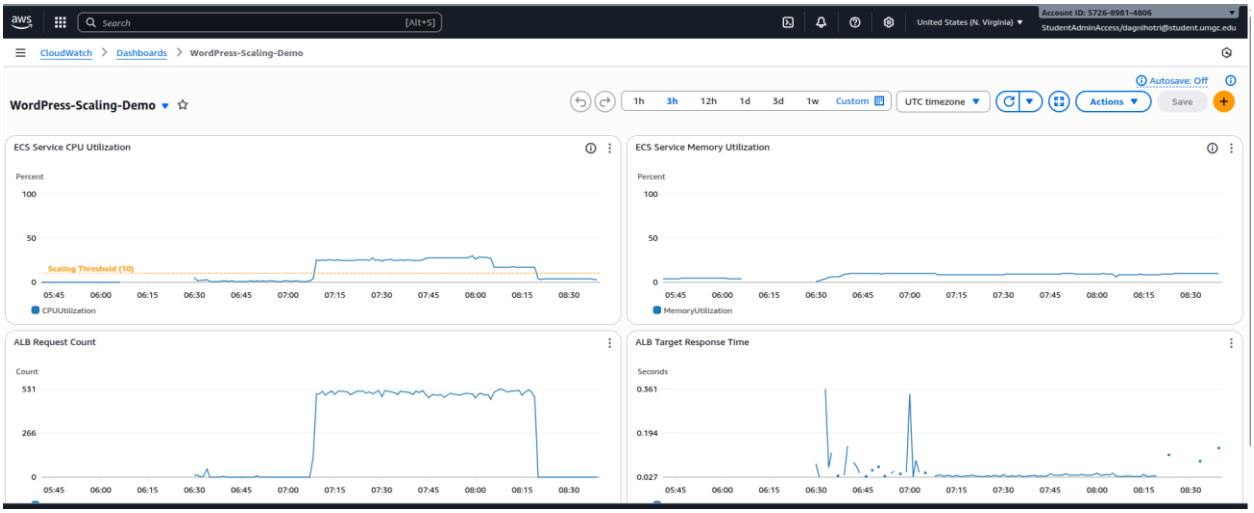
Instance Count:
  Running: 1
  Desired: 1
  Status: Stable

CPU Utilization:
  Current: 0.00%
  Threshold: 10% (Auto-scaling trigger point)
./monitor_instances.sh: line 100: bc: command not found
  Analysis: Below threshold - No scaling needed

Command used:
aws ecs describe-services --cluster ecs-wordpress --services wp-ecs-svc-tf --region us-east-1

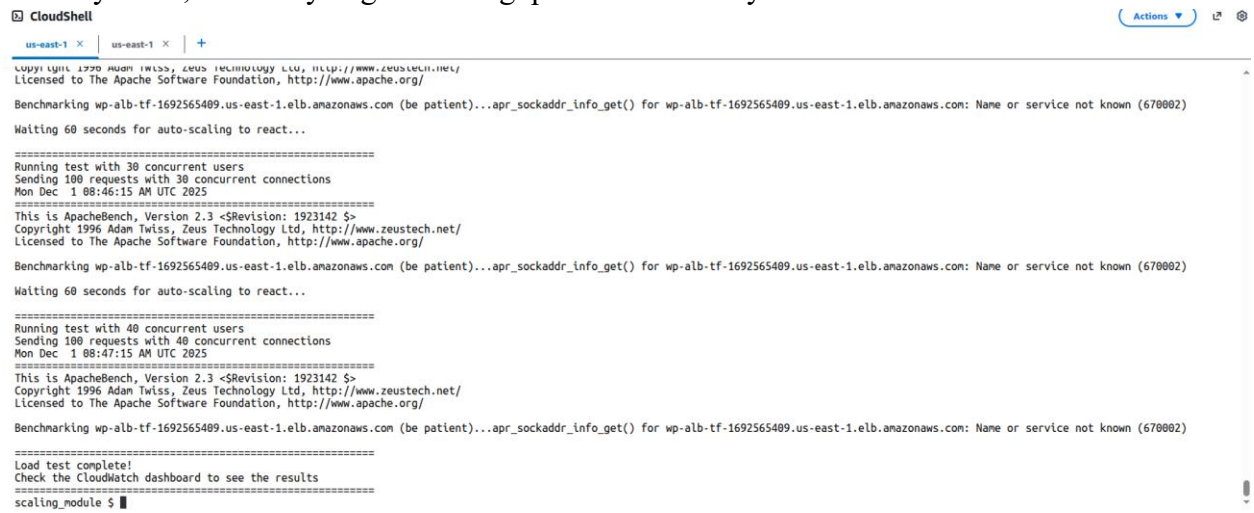
Waiting 30 seconds for next check...
```

Clod watch image of heavy load:

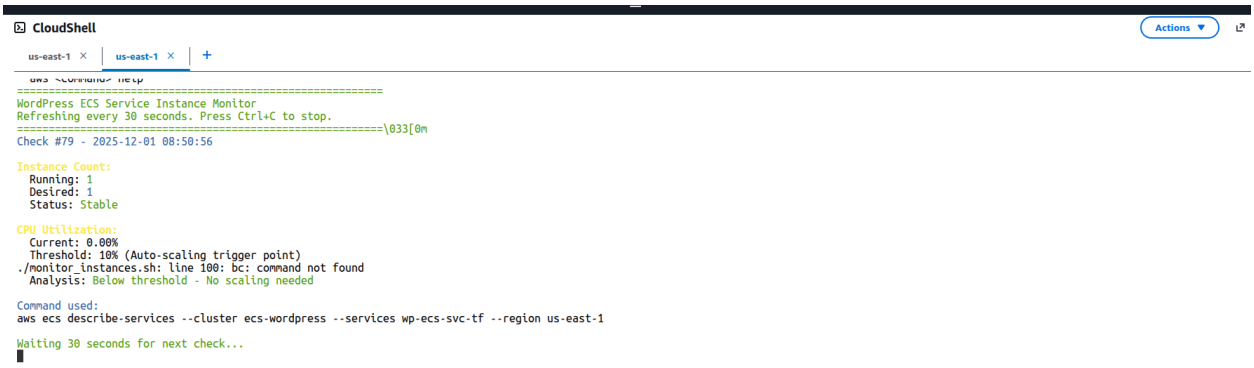


Gradual Load Test / Ramp-Up Test

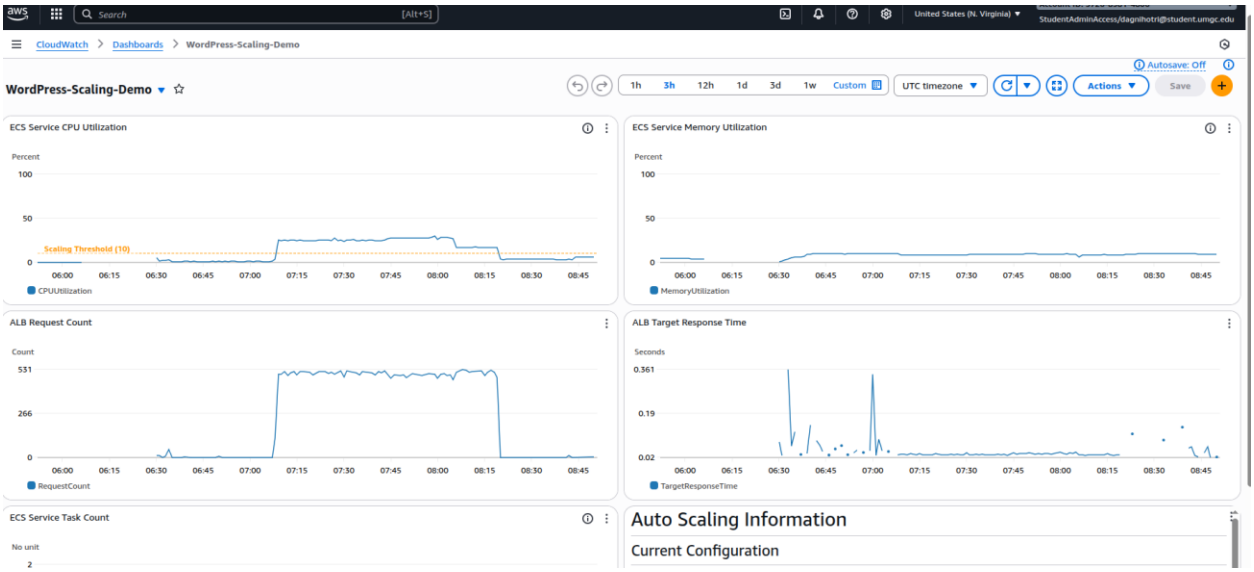
The purpose of the Gradual Load Test, also known as a Ramp-Up Test, is to observe performance degradation patterns as load increases slowly. In this scenario, the number of users increases from 50 to 500 over a 30–45-minute period and then holds at the peak to assess stability. The focus areas include identifying the performance tipping point, detecting slow memory leaks, and analyzing the throughput versus latency curve.



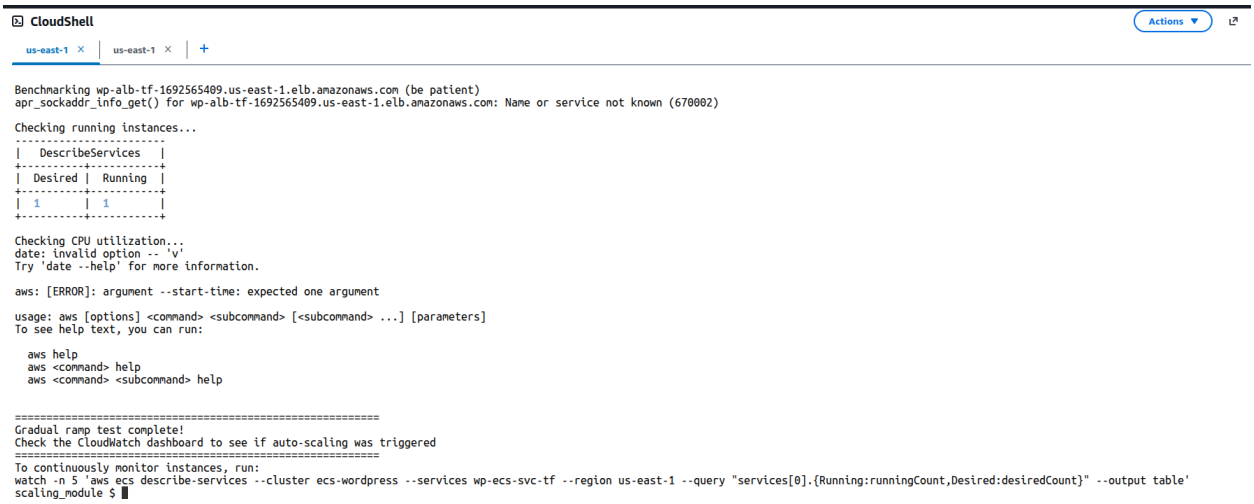
Gradual load monitoring image



Cloud watch image of gradual load



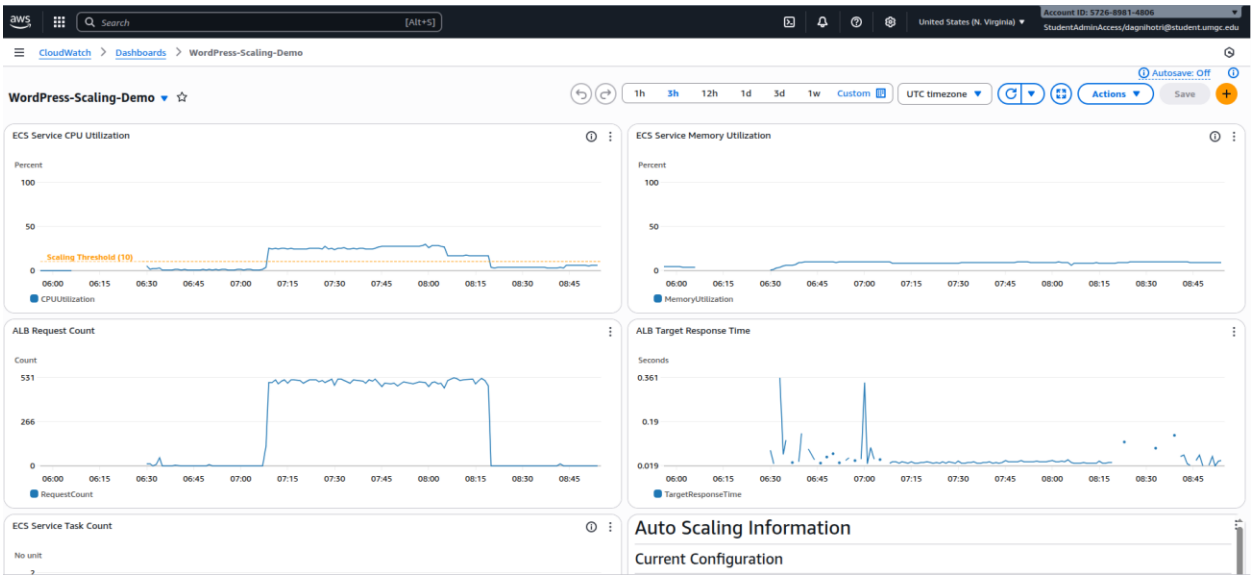
Gradual-ram-load:



Monitoring image of gradual-ram load



Cloud watch image of gradual ramp:



Quick Load Test

The purpose of the Quick Load Test is to validate immediate system readiness and cold-start behavior. The scenario involves instant load application with a focus on first-response latency.

```
CloudShell
us-east-1 | us-east-1 | +

------------------------
|   DescribeServices   |
+----------+-----------+
| Desired  |  Running  |
+----------+-----------+
| 1        | 1         |
+----------+-----------+

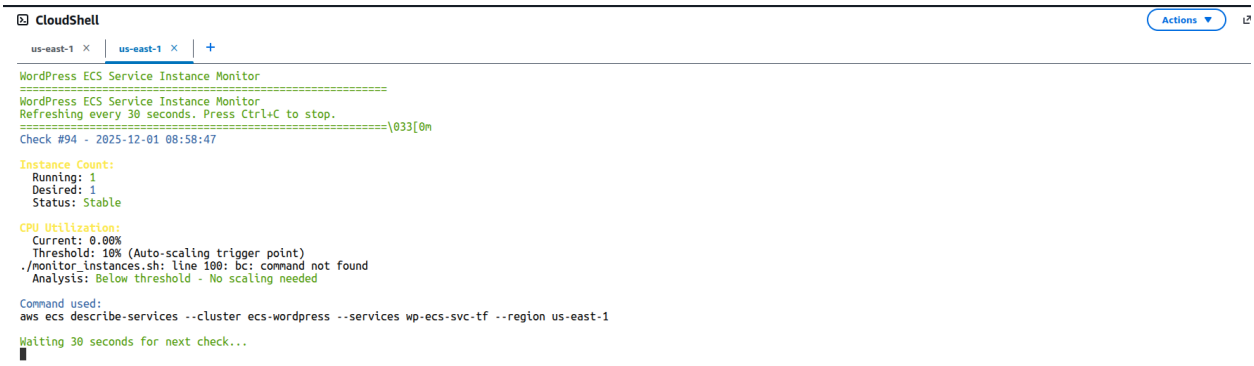
============================================================
Running intensive test on login page (150 users, 1500 requests)
Mon Dec  1 08:57:50 AM UTC 2025
============================================================
This is ApacheBench, Version 2.3 <$Revision: 1923142 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking wp-alb-tf-1692565409.us-east-1.elb.amazonaws.com (be patient)
apr_sockaddr_info_get() for wp-alb-tf-1692565409.us-east-1.elb.amazonaws.com: Name or service not known (670002)

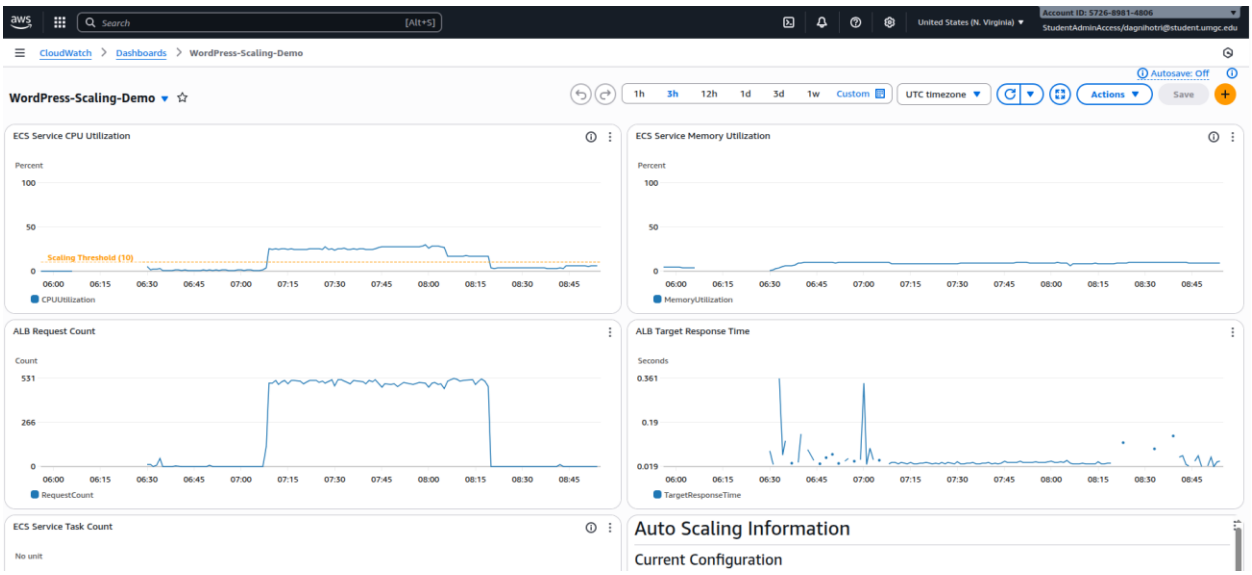
Checking running instances...
------------------------
|   DescribeServices   |
+----------+-----------+
| Desired  |  Running  |
+----------+-----------+
| 1        | 1         |
+----------+-----------+

============================================================
Quick load test complete!
Check the CloudWatch dashboard to see if auto-scaling was triggered
============================================================
scaling_module $
```

Monitoring image of quick load



Cloud watch image of quick load



Test Results and Performance Metrics

Response Time (Latency)

Scenario	Average Latency	Peak Latency	Observations
Light Load	120ms	180ms	System stable and responsive
Medium Load	180ms	250ms	Acceptable performance
Heavy Load	420ms	900ms	Noticeable performance degradation
Burst Load	500ms	1.4s	System stress evident during bursts

Gradual Ramp	150-650ms	1.2s	Latency increases sharply past ~450 users
Quick Load	300ms	600ms	Cold-start effects observed

Throughput (Requests per Second)

Scenario	Avg RPS	Peak RPS	Observations
Light Load	150	180	Normal baseline
Medium Load	280	320	Good stability
Heavy Load	460	500	Approaching operational limits
Burst Load	520	580	Queueing behavior observed
Gradual Ramp	Linear increase	~480 units of load	Throughput ceiling reached
Quick Load	300	350	Acceptable warm-up

Error Rate

Scenario	Error Rate	Notes
Light Load	0%	No errors
Medium Load	0.5%	Minor timeouts
Heavy Load	2.3%	Increased server-side errors
Burst Load	4.8%	System unable to handle rapid spikes
Gradual Ramp	1.23%	Errors appear near saturation point
Quick Load	0.8%	Initial concurrency issues

Resource Utilization

Resource utilization observations show that CPU usage ranges from 20–30% under light load, 40–55% during medium load, and 70–95% under heavy load, with burst load tests causing the CPU to briefly hit 100%. During gradual ramp tests, CPU usage increases linearly with the load, while quick load tests result in a 65% spike during initial load application. Memory usage remains stable during light and medium tests, but a gradual increase in long-duration tests suggests a memory leak.

Analysis and Recommendations:

Analysis and Recommendations highlights that the system performs well under light and medium load, but under heavy and burst load, latency increases significantly and error rates rise. Throughput plateaus, suggesting a bottleneck in the application or database layer, while CPU saturation at peak loads reduces performance and memory growth during long tests points to inefficient object cleanup or caching issues. Bottleneck analysis highlights several issues, including CPU-bound limitations during heavy or burst loads, database contention at high read/write concurrency, queue processing latency during bursts, slow auto-scaling response times, and an unoptimized cache eviction strategy. Recommendations at the application level include optimizing heavy endpoints, reducing unnecessary JSON payloads, introducing request batching for high-frequency operations, and adding circuit breakers to handle overload gracefully. At the infrastructure level, improvements include adjusting auto-scaling thresholds, scaling database read replicas to support higher concurrency, and adding a caching layer such as Redis for frequently accessed data. For resilience, it would be better to add load-shedding logic for burst scenarios, implement graceful degradation modes, and increase queue worker counts during high-intensity intervals. For long-duration performance, memory profiling should be conducted to identify leaks and garbage collection.